Date Picker / Calendar

Problem: Build a date picker: an input field that opens a month calendar, lets the user pick a date with the mouse or the keyboard, and respects min / max dates.
Often asked at: Atlassian, Flipkart, MakeMyTrip
Why it matters: date math and keyboard navigation
Tech stack: React (Vite, JavaScript) *(default, not specified)*
Time limit: 60 minutes *(default, not specified)*

PART A: Thought process

The general steps for any machine coding task

1. **Understand the requirements:** functional, non-functional, clarifying questions, assumptions.
2. **List the interactions:** every action and its expected result. This becomes your checklist.
3. **Design the architecture:** components and their single responsibility.
4. **Design the state:** the smallest state that can describe the UI; derive the rest.
5. **Design data and contracts:** data shapes, component props, API payloads.
6. **Trace the data flow** of one key interaction end to end.
7. **Find the hard parts** early and decide how to solve them.
8. **Build in layers:** simplest working version first, then add one working layer at a time.
9. **Cover states and edge cases.**
10. **Wrap up:** trade-offs and what you would add with more time.

Time budget for 60 minutes

Phase	Time
Requirements, clarifying questions, assumptions	5 min
Interactions, architecture, state, contracts	8 min
Date utils (grid, add days / months, parse)	10 min
Calendar grid + month navigation + selection	12 min
Popover input, outside click, Escape	8 min
Keyboard navigation + focus management	10 min
min / max, typed input validation, polish	5 min
Wrap-up and trade-offs	2 min

Tip: think out loud

Say what you are deciding and why: "*I'll keep the value as a* `YYYY-MM-DD` *string so I never fight timezones*", "*I'll derive the visible month from the focused day so there's one source of truth*". The interviewer grades your

reasoning as much as the code. If you run out of time, a clear plan for the missing part still earns credit.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- A text input with a calendar button; clicking the button opens a popover calendar.
- The calendar shows one month as a 7 x 6 grid with weekday headers and the month / year title.
- Previous / next month buttons.
- Click a day to select it: the input shows the date, the popover closes.
- Highlight **today**, the **selected** day, and mute days of the adjacent months.
- **min / max** dates: days outside the range are disabled; navigation cannot go past them.
- **Keyboard navigation:** arrows move by day / week, PageUp / PageDown by month (Shift for year), Home / End to start / end of week, Enter / Space selects, Escape closes.
- The user can also **type** a date (`YYYY-MM-DD`); invalid or out-of-range input shows an error.
- Close on outside click.

Non-functional requirements

Area	What it means here
Correctness	No timezone off-by-one, correct leap years, month overflow (Jan 31 + 1 month)
Performance	42 cells only; memoise the grid per month; no heavy library
Error handling	Invalid typed text / impossible dates (Feb 31) show a message, never crash
Accessibility	ARIA grid pattern, roving tabindex, `aria-selected` , `aria-current="date"` , full-date `aria-label` , focus returns to the trigger
Reusability	Controlled component: `value` / `onChange` , like a native input
Extensibility	`locale` , `weekStartsOn` , `min` / `max` as props; utils reusable for a range picker

Clarifying questions

1. Single date or a date **range** (check-in / check-out like MakeMyTrip)?
2. What format does the parent want: a `Date` , an ISO string, a timestamp? Does time of day matter?
3. Which week start and locale? (India / US: Sunday or Monday? Month names localised?)
4. Should users be able to **type** the date, or only pick it?
5. Any disabled dates beyond min / max (holidays, sold-out days)?
6. Is a native `<input type="date">` acceptable, or is a custom UI required?

Assumptions we will build with

- **Single date** picker; range is an extension discussed at the end.
- The value is a `'YYYY-MM-DD'` **string** (a calendar date, no time, no timezone). `null` means empty.
- Controlled component: the parent owns the value.
- `min` / `max` are optional `'YYYY-MM-DD'` strings, inclusive.
- Typing is allowed in strict `YYYY-MM-DD`; display formatting of the grid uses `Intl` with an optional `locale`.
- `weekStartsOn` prop: 0 = Sunday (default), 1 = Monday.
- No external date library (show you can do the math); mention `date-fns` as the production choice.

---

2. Identify interactions / use cases

Action / input	Expected behavior / output
Click the calendar button	Popover opens on the selected month (or today's), focus goes to the selected / today cell
Click a day	`onChange('YYYY-MM-DD')`, input shows it, popover closes, focus back on the button
Click a muted day of the next month	Selects that date (same as any day)
Click ‹ / ›	Month changes, focus stays on the button
‹ when the previous month is fully before `min`	Button disabled
ArrowLeft / ArrowRight	Focus moves one day; crossing the month edge switches month
ArrowUp / ArrowDown	Focus moves one week
Home / End	Start / end of the week (respects `weekStartsOn`)
PageUp / PageDown	Same day previous / next month (Jan 31 → Feb 28)
Shift + PageUp / PageDown	Same day previous / next year
Arrow towards a date beyond `min` / `max`	Focus clamps to the boundary date
Enter / Space on a focused day	Selects it (native button click)
Escape	Popover closes, focus returns to the calendar button
Click outside / Tab out	Popover closes
Type `2026-12-25` + Enter / blur	Valid and in range → `onChange`; calendar opens on December next time
Type `2026-02-31`	Error "Use the format YYYY-MM-DD"
Type a date before `min`	Error "Pick a date between …"
Clear the input + blur	`onChange(null)`
Today	Bold ring + `aria-current="date"`

3. Design the architecture

Module	Responsibility
`utils/date.js`	Pure date math: parse / format ISO, add days / months, month grid, range checks. No React.
`Calendar`	Renders one month grid, owns the focused day, handles keyboard navigation, reports `onSelect(date)`
`DatePicker`	Input + trigger button + popover; owns open state, typed text, validation, outside click / Escape
`App`	Demo: holds the value, passes `min` / `max`

```
App  (value state)
└── DatePicker  (open, text, error)
    ├── <input>  +  <button 📅>
    └── Popover (role="dialog")
        └── Calendar  (focused day)
            ├── Header: ‹  "November 2026"  ›
            └── <table role="grid">  7 x 6 day buttons
                        ▲
                utils/date.js (pure functions)
```

Why split this way: `Calendar` is useful on its own (inline calendar on a booking page) and `DatePicker` is just the popover shell around it. All date math lives in pure functions, so the trickiest code is unit-testable without a browser. A separate `DayCell` component would add props plumbing for no gain at this size, so the cell is inline.

4. Design the state / data model

State or field	Owner	Purpose
`value` (`'YYYY-MM-DD'` | `null`)	Parent (`App`)	The selected date, single source of truth
`open`	`DatePicker`	Is the popover visible
`text`	`DatePicker`	Draft text in the input while typing
`error`	`DatePicker`	Validation message for typed text
`focused` (`Date`)	`Calendar`	The day with the roving tabindex; **the visible month is derived from it**

```
// App
const [date, setDate] = useState(null);          // 'YYYY-MM-DD' | null
```

```
// DatePicker
const [open, setOpen] = useState(false);
const [text, setText] = useState(value ?? '');
const [error, setError] = useState('');

// Calendar
const [focused, setFocused] = useState(() => clamp(selected ?? today, min, max));
const year = focused.getFullYear();              // derived
const month = focused.getMonth();                // derived
const days = useMemo(() => getMonthGrid(year, month, weekStartsOn), [year, month, weekStartsOn]);
```

Senior note: derive, don't store.

- No `viewMonth` state: it is `focused.getMonth()`. Two states (`viewMonth` + `focusedDate`) can drift apart: the user pages to March while focus is still on a February day. One state makes that impossible.
- The 42 grid days, the title, weekday labels, disabled flags and `isToday` are all computed during render.
- `selected` inside `Calendar` is not state, it is a prop derived from the parent's `value`.
- `text` is the only "copy" of `value`, and only because a half-typed string (`2026-1`) is not a valid value yet. It is reset whenever `value` changes.

---

5. Design the data structures and contracts

Example data

```
value = '2026-11-02';          // what the parent stores and sends to an API
min   = '2026-11-02';          // today: no bookings in the past
max   = '2027-11-02';          // one year ahead

// internally: local-midnight Date objects
new Date(2026, 10, 2)          // month is 0-based!
```

Why a string and not a `Date`? A booking date is a **calendar date**, not an instant. `new Date('2026-11-02')` is parsed as **UTC midnight**, which is Nov 1 in New York. A string has no timezone, compares correctly with `<` / `===`, serialises to JSON unchanged, and is what most APIs expect. Cells use `toISO(date)` as a stable React key.

Component contracts

```
<DatePicker
  value={'2026-11-02' | null}          // controlled
  onChange={(iso: string | null) => {}}
  min="2026-11-02"                     // optional, inclusive
  max="2027-11-02"                     // optional, inclusive
  label="Travel date"
```

```
  weekStartsOn={0 | 1}                // default 0 (Sunday)
  locale="en-IN"                      // optional, Intl locale
/>

<Calendar
  selected={Date | null}
  onSelect={(date: Date) => {}}
  min={Date | null} max={Date | null}
  weekStartsOn={0} locale="en-IN" autoFocus
/>
```

Utils contract (pure functions)

Function	Input → Output
`fromISO('2026-02-31')`	`null` (invalid or impossible) / local `Date`
`toISO(date)`	`'2026-11-02'`
`addDays(date, n)` / `addMonths(date, n)`	new `Date` (month clamps to last day)
`getMonthGrid(2026, 10, 1)`	42 `Date`s starting Mon 26 Oct 2026
`isOutOfRange(date, min, max)` / `clamp(date, min, max)`	boolean / `Date`

There is no backend. If disabled dates came from an API (sold-out days), the contract would be `GET /availability?month=2026-11` → `{ "disabled": ["2026-11-14", "2026-11-15"] }`, fetched per visible month and passed as an `isDateDisabled(date)` prop.

---

6. Trace the data flow

Key interaction: keyboard user opens the picker, presses PageDown, then Enter.

```
Click / Enter on 📅 button
   |
   ▼
DatePicker: setOpen(true)  ──►  popover renders <Calendar autoFocus>
   |
   ▼
Calendar init: focused = clamp(selected ?? today, min, max)   // 2 Nov 2026
shouldFocusCell.current = true  ──►  effect: grid.querySelector('[tabindex="0"]').focus()
   |
   ▼
PageDown keydown on the grid
   |
   ▼
handleKeyDown: KEY_MOVES.PageDown(focused) = addMonths(2 Nov, 1) = 2 Dec 2026
               clamp to [min, max]; shouldFocusCell.current = true; setFocused(2 Dec)
```

```
   |
   ▼
Re-render: month derived = December, getMonthGrid(2026, 11) (memo miss → recompute)
           2 Dec gets tabIndex=0
   |
   ▼
Effect [focused]: focus the tabIndex=0 button  →  screen reader reads "Wednesday, 2 December 2026"
   |
   ▼
Enter → native button click → onSelect(2 Dec)
   |
   ▼
DatePicker: onChange(toISO(d)) = '2026-12-02'; setOpen(false); focus 🗓 button
   |
   ▼
App: setDate('2026-12-02')  ⟶  DatePicker sees value !== prevValue → text = '2026-12-02'
```

---

7. Important technical challenges

Challenge 1: Building the month grid

Problem: November 2026 starts on a Sunday; with a Monday week start, the first row must begin with Mon 26 Oct. Months span 4, 5 or 6 rows, so the popover height jumps while paging.

Solution: compute the offset of the 1st from the week start, then always render **42 cells** (6 weeks). `new Date(y, m, 1 - offset + i)` lets JavaScript roll negative days into the previous month and days past the end into the next one.

```
export function getMonthGrid(year, month, weekStartsOn = 0) {
  const offset = (new Date(year, month, 1).getDay() - weekStartsOn + 7) % 7;
  return Array.from({ length: 42 }, (_, i) => new Date(year, month, 1 - offset + i));
}
```

`+ 7) % 7` keeps the offset positive when the 1st is a Sunday (day 0) and the week starts on Monday (1).

Challenge 2: Date math pitfalls (timezone, overflow, DST)

Problem:

- `new Date('2026-11-02')` is **UTC**, so in America it shows Nov 1 (off by one).
- `setMonth(month + 1)` on **Jan 31** gives **Mar 3** (Feb 31 overflows).
- Adding `24 * 60 * 60 * 1000` ms crosses a DST change at 23 or 25 hours, so you land on the same day or skip one.
- `'2026-02-31'` must be rejected, not rolled over.

Solution: parse manually into **local** `new Date(y, m, d)`, always add days through the date constructor, clamp the day when adding months, and verify the parsed parts round-trip.

```
export function fromISO(str) {
  const m = /^(\d{4})-(\d{2})-(\d{2})$/.exec(str ?? '');
  if (!m) return null;
  const [y, mo, d] = [+m[1], +m[2] - 1, +m[3]];
  const date = new Date(y, mo, d);
  return date.getMonth() === mo && date.getDate() === d ? date : null; // rejects Feb 31
}

export const addDays = (d, n) => new Date(d.getFullYear(), d.getMonth(), d.getDate() + n);

export function addMonths(d, n) {
  const lastDay = new Date(d.getFullYear(), d.getMonth() + n + 1, 0).getDate(); // day 0 = last day of
prev month
  return new Date(d.getFullYear(), d.getMonth() + n, Math.min(d.getDate(), lastDay));
}
```

Challenge 3: Keyboard navigation and focus management

Problem: 42 tab stops would make Tab unusable. Arrow keys must move focus, also across month boundaries (ArrowRight on Nov 30 → Dec 1 renders a new grid and the old focused button may be gone). But clicking ‹ › must **not** pull focus into the grid.

Solution:

- **Roving tabindex:** only the focused day has `tabIndex={0}`; all others `-1`. Tab enters the grid on one day and leaves it in one step.
- One `KEY_MOVES` map instead of a long `switch`; every move goes through `clamp(min, max)`.
- Focus is moved in an **effect after render** (the target cell exists only after the new month renders), and only when a ref flag says the change came from the keyboard.

```
const KEY_MOVES = {
  ArrowLeft: (d) => addDays(d, -1),   ArrowRight: (d) => addDays(d, 1),
  ArrowUp:   (d) => addDays(d, -7),   ArrowDown:  (d) => addDays(d, 7),
  Home: (d, ws) => startOfWeek(d, ws),
  End:  (d, ws) => addDays(startOfWeek(d, ws), 6),
  PageUp:   (d, ws, shift) => addMonths(d, shift ? -12 : -1),
  PageDown: (d, ws, shift) => addMonths(d, shift ? 12 : 1),
};

function handleKeyDown(e) {
  const move = KEY_MOVES[e.key];
  if (!move) return;                    // Enter / Space → native button click
  e.preventDefault();                   // stop page scroll on arrows / PageDown
  shouldFocusCell.current = true;
```

```
  setFocused(clamp(move(focused, weekStartsOn, e.shiftKey), min, max));
}

useEffect(() => {
  if (!shouldFocusCell.current) return;
  shouldFocusCell.current = false;
  gridRef.current?.querySelector('button[tabindex="0"]')?.focus();
}, [focused]);
```

Challenge 4: Popover lifecycle (outside click, Escape, focus return, typed input)

Problem: the popover must close on outside click, on Escape and on Tab-out, and return focus to the trigger so keyboard users aren't dropped at the top of the page. Edge case found while testing: on the earliest allowed month, clicking ‹ disables the button you just clicked, focus falls to `<body>`, and an `onKeyDown` on the popover no longer receives Escape.

Solution: while open, listen on `document` for `pointerdown` (outside check with `ref.contains`) **and** `keydown` Escape; clean both up when it closes. Typed text is a draft committed on Enter / blur, and reset when `value` changes (state adjusted during render, no effect).

```
useEffect(() => {
  if (!open) return;
  const onPointerDown = (e) => { if (!rootRef.current.contains(e.target)) setOpen(false); };
  const onKeyDown = (e) => {
    if (e.key !== 'Escape') return;
    setOpen(false);
    toggleRef.current.focus();
  };
  document.addEventListener('pointerdown', onPointerDown);
  document.addEventListener('keydown', onKeyDown);
  return () => {
    document.removeEventListener('pointerdown', onPointerDown);
    document.removeEventListener('keydown', onKeyDown);
  };
}, [open]);
```

---

8. Implement in the correct order

#	Step	Working result
1	Vite app, `App` with `value` state, plain `<input>`	Shows / edits a string
2	`utils/date.js`: `toISO`, `fromISO`, `addDays`, `getMonthGrid`	Pure functions, test with `node` asserts
3	`Calendar`: static grid for the current month, weekday headers	A month renders, 6 rows

#	Step	Working result
4	Click a day → `onSelect`; selected / today / outside classes	Picking works inline
5	‹ › buttons via `addMonths` (with day clamping)	Month navigation
6	`DatePicker`: trigger button, popover, close on select	Real picker
7	Outside click + Escape (document listeners), focus back to trigger	Popover lifecycle done
8	Roving tabindex + `KEY_MOVES` + focus effect	Full keyboard support
9	`min` / `max`: disable cells, clamp moves, disable ‹ ›	Range respected
10	Typed input: draft text, commit on Enter / blur, errors	Typing works
11	ARIA: grid roles, `aria-selected`, `aria-current`, `aria-label`, `aria-expanded`	Screen reader friendly
12	`locale`, `weekStartsOn`, CSS polish	Configurable and clean

Steps 1–6 already make a usable demo at ~25 minutes; everything after is a layer on a working base.

---

9. Cover all states and edge cases

State / edge case	Handling
Initial, no value	Input empty with placeholder; calendar opens on today (clamped into range)
Initial with value	Input shows it; calendar opens on that month with it focused
Popover open / closed	`open` state; `aria-expanded` on the trigger
Selected	Filled circle, `aria-selected="true"` on the cell
Today	Ring + bold + `aria-current="date"`
Days of adjacent months	Muted but clickable
Out of range (`min` / `max`)	`disabled` buttons; ‹ › disabled; keyboard moves clamp
`min` > today	Calendar opens on `min`, not on a fully disabled month
Invalid typed text (`abc`, `2026-13-01`, `2026-02-31`)	`aria-invalid`, error with `role="alert"`, value unchanged
Typed date out of range	Range error message
Cleared input	`onChange(null)`
Jan 31 + PageDown	Feb 28 (or Feb 29 in leap years)
Leap year	`2024-02-29` valid, `2026-02-29` invalid
Year boundary	Dec 31 + ArrowRight → Jan 1 next year
DST change	No ms arithmetic, so no skipped / duplicated days

State / edge case	Handling
Timezone of user	Value is a string, parsed as local: no off-by-one
Month starting on week start day	Offset 0, grid still 42 cells
Focus lost to `<body>` after ‹ disables itself	Escape still works (document listener)
Loading / error from API	Not applicable (no backend); with availability API: show a spinner over the grid and keep navigation enabled

---

10. Final structure

```
_62_date_picker_calendar/
├── index.html
├── package.json
├── vite.config.js
└── src/
    ├── main.jsx
    ├── App.jsx                 // demo: value state, min / max
    ├── styles.css
    ├── components/
    │   ├── DatePicker.jsx      // input + popover shell
    │   └── Calendar.jsx        // month grid + keyboard navigation
    └── utils/
        ├── date.js             // pure date math
        └── date.test.js        // node assert self-check
```

---

11. Full working code

`src/utils/date.js`

```
// All dates are local-midnight Date objects; the public value is a 'YYYY-MM-DD' string (no timezone).
const pad = (n) => String(n).padStart(2, '0');

export const toISO = (d) => `${d.getFullYear()}-${pad(d.getMonth() + 1)}-${pad(d.getDate())}`;

export function fromISO(str) {
  const m = /^(\d{4})-(\d{2})-(\d{2})$/.exec(str ?? '');
  if (!m) return null;
  const [y, mo, d] = [+m[1], +m[2] - 1, +m[3]];
  const date = new Date(y, mo, d); // local time, unlike new Date('2026-11-02') which is UTC
  // reject rollovers like 2026-02-31 -> Mar 3
```

```
  return date.getMonth() === mo && date.getDate() === d ? date : null;
}

export const startOfToday = () => {
  const n = new Date();
  return new Date(n.getFullYear(), n.getMonth(), n.getDate());
};

export const isSameDay = (a, b) =>
  !!a && !!b &&
  a.getFullYear() === b.getFullYear() && a.getMonth() === b.getMonth() && a.getDate() === b.getDate();

// new Date(y, m, d + n) handles month/year overflow and DST (no ms arithmetic)
export const addDays = (d, n) => new Date(d.getFullYear(), d.getMonth(), d.getDate() + n);

// Jan 31 + 1 month = Feb 28/29, not Mar 3
export function addMonths(d, n) {
  const lastDay = new Date(d.getFullYear(), d.getMonth() + n + 1, 0).getDate();
  return new Date(d.getFullYear(), d.getMonth() + n, Math.min(d.getDate(), lastDay));
}

export const startOfWeek = (d, weekStartsOn = 0) => addDays(d, -((d.getDay() - weekStartsOn + 7) %
7));

export const isOutOfRange = (d, min, max) => !!((min && d < min) || (max && d > max));

export const clamp = (d, min, max) => (min && d < min ? min : max && d > max ? max : d);

// 6 rows x 7 cols = 42 cells so the popover height never jumps between months
export function getMonthGrid(year, month, weekStartsOn = 0) {
  const offset = (new Date(year, month, 1).getDay() - weekStartsOn + 7) % 7;
  return Array.from({ length: 42 }, (_, i) => new Date(year, month, 1 - offset + i));
}

export function getWeekdayLabels(locale, weekStartsOn = 0) {
  const fmt = new Intl.DateTimeFormat(locale, { weekday: 'short' });
  // 1 Jan 2023 was a Sunday
  return Array.from({ length: 7 }, (_, i) => fmt.format(new Date(2023, 0, 1 + ((i + weekStartsOn) %
7))));
}
```

src/components/Calendar.jsx

```
import { useEffect, useId, useMemo, useRef, useState } from 'react';
import {
  addDays, addMonths, clamp, getMonthGrid, getWeekdayLabels,
```

```
  isOutOfRange, isSameDay, startOfToday, startOfWeek, toISO,
} from '../utils/date';

const KEY_MOVES = {
  ArrowLeft: (d) => addDays(d, -1),
  ArrowRight: (d) => addDays(d, 1),
  ArrowUp: (d) => addDays(d, -7),
  ArrowDown: (d) => addDays(d, 7),
  Home: (d, ws) => startOfWeek(d, ws),
  End: (d, ws) => addDays(startOfWeek(d, ws), 6),
  PageUp: (d, ws, shift) => addMonths(d, shift ? -12 : -1),
  PageDown: (d, ws, shift) => addMonths(d, shift ? 12 : 1),
};

export default function Calendar({ selected, onSelect, min, max, weekStartsOn = 0, locale, autoFocus =
false }) {
  const today = startOfToday();
  // The one piece of state: the visible month is derived from the focused day.
  const [focused, setFocused] = useState(() => clamp(selected ?? today, min, max));
  const shouldFocusCell = useRef(autoFocus);
  const gridRef = useRef(null);
  const titleId = useId();

  // Move DOM focus only after keyboard navigation, so prev/next buttons keep their own focus.
  useEffect(() => {
    if (!shouldFocusCell.current) return;
    shouldFocusCell.current = false;
    gridRef.current?.querySelector('button[tabindex="0"]')?.focus();
  }, [focused]);

  const year = focused.getFullYear();
  const month = focused.getMonth();
  const days = useMemo(() => getMonthGrid(year, month, weekStartsOn), [year, month, weekStartsOn]);
  const weekdays = useMemo(() => getWeekdayLabels(locale, weekStartsOn), [locale, weekStartsOn]);
  const title = new Intl.DateTimeFormat(locale, { month: 'long', year: 'numeric' }).format(focused);
  const fullDate = new Intl.DateTimeFormat(locale, { dateStyle: 'full' });

  const prevDisabled = !!min && new Date(year, month, 0) < min; // last day of prev month
  const nextDisabled = !!max && new Date(year, month + 1, 1) > max; // first day of next month

  const goMonth = (n) => setFocused(clamp(addMonths(focused, n), min, max));

  function handleKeyDown(e) {
    const move = KEY_MOVES[e.key];
    if (!move) return; // Enter / Space fall through to the native button click
    e.preventDefault();
    shouldFocusCell.current = true;
```

```
    setFocused(clamp(move(focused, weekStartsOn, e.shiftKey), min, max));
  }

  const weeks = Array.from({ length: 6 }, (_, r) => days.slice(r * 7, r * 7 + 7));

  return (
    <div className="calendar">
      <div className="calendar__header">
        <button type="button" onClick={() => goMonth(-1)} disabled={prevDisabled} aria-label="Previous
month">‹</button>
        <h2 id={titleId} aria-live="polite">{title}</h2>
        <button type="button" onClick={() => goMonth(1)} disabled={nextDisabled} aria-label="Next
month">›</button>
      </div>

      <table role="grid" aria-labelledby={titleId} ref={gridRef} onKeyDown={handleKeyDown}>
        <thead>
          <tr>{weekdays.map((w, i) => <th key={i} scope="col" abbr={w}>{w}</th>)}</tr>
        </thead>
        <tbody>
          {weeks.map((week, r) => (
            <tr key={r}>
              {week.map((d) => {
                const isSelected = isSameDay(d, selected);
                const classes = [
                  'day',
                  d.getMonth() !== month && 'day--outside',
                  isSameDay(d, today) && 'day--today',
                  isSelected && 'day--selected',
                ].filter(Boolean).join(' ');
                return (
                  <td key={toISO(d)} role="gridcell" aria-selected={isSelected}>
                    <button
                      type="button"
                      className={classes}
                      tabIndex={isSameDay(d, focused) ? 0 : -1}
                      disabled={isOutOfRange(d, min, max)}
                      aria-label={fullDate.format(d)}
                      aria-current={isSameDay(d, today) ? 'date' : undefined}
                      onClick={() => { setFocused(d); onSelect(d); }}
                    >
                      {d.getDate()}
                    </button>
                  </td>
                );
              })}
            </tr>
```

```
          ))}
        </tbody>
      </table>
    </div>
  );
}
```

`src/components/DatePicker.jsx`

```
import { useEffect, useId, useMemo, useRef, useState } from 'react';
import Calendar from './Calendar';
import { fromISO, isOutOfRange, toISO } from '../utils/date';

// Controlled: value / min / max are 'YYYY-MM-DD' strings, onChange receives a string or null.
export default function DatePicker({ value, onChange, min, max, label = 'Date', weekStartsOn = 0,
locale }) {
  const [open, setOpen] = useState(false);
  const [text, setText] = useState(value ?? '');
  const [error, setError] = useState('');
  const [prevValue, setPrevValue] = useState(value);
  const rootRef = useRef(null);
  const toggleRef = useRef(null);
  const id = useId();

  const minDate = useMemo(() => fromISO(min), [min]);
  const maxDate = useMemo(() => fromISO(max), [max]);

  // Parent (or the calendar) changed the value: reset the draft text.
  if (value !== prevValue) {
    setPrevValue(value);
    setText(value ?? '');
    setError('');
  }

  // Close on a click outside, or Escape anywhere (focus can drop to <body> when a nav button disables
itself).
  useEffect(() => {
    if (!open) return;
    const onPointerDown = (e) => {
      if (!rootRef.current.contains(e.target)) setOpen(false);
    };
    const onKeyDown = (e) => {
      if (e.key !== 'Escape') return;
      setOpen(false);
      toggleRef.current.focus();
    };
```

```
    document.addEventListener('pointerdown', onPointerDown);
    document.addEventListener('keydown', onKeyDown);
    return () => {
      document.removeEventListener('pointerdown', onPointerDown);
      document.removeEventListener('keydown', onKeyDown);
    };
  }, [open]);

  function close() {
    setOpen(false);
    toggleRef.current.focus(); // return focus to where the popover was opened from
  }

  function commitText() {
    if (text === value || (text === '' && value == null)) return;
    if (text === '') { setError(''); onChange(null); return; }
    const d = fromISO(text);
    if (!d) return setError('Use the format YYYY-MM-DD');
    if (isOutOfRange(d, minDate, maxDate)) return setError(`Pick a date between ${min ?? 'any'} and
${max ?? 'any'}`);
    setError('');
    onChange(text);
  }

  // Tabbing to something outside closes the popover (relatedTarget is null for clicks on plain text).
  function handleBlur(e) {
    if (e.relatedTarget && !rootRef.current.contains(e.relatedTarget)) setOpen(false);
  }

  return (
    <div className="datepicker" ref={rootRef} onBlur={handleBlur}>
      <label htmlFor={id}>{label}</label>
      <div className="datepicker__field">
        <input
          id={id}
          value={text}
          placeholder="YYYY-MM-DD"
          autoComplete="off"
          aria-invalid={!!error}
          aria-describedby={error ? `${id}-error` : undefined}
          onChange={(e) => { setText(e.target.value); setOpen(false); }}
          onBlur={commitText}
          onKeyDown={(e) => e.key === 'Enter' && commitText()}
        />
        <button
          ref={toggleRef}
          type="button"
```

```
          aria-label="Choose date"
          aria-haspopup="dialog"
          aria-expanded={open}
          onClick={() => setOpen((o) => !o)}
        >
          
        </button>
      </div>
      {error && <p id={`${id}-error`} className="datepicker__error" role="alert">{error}</p>}

      {open && (
        <div className="datepicker__popover" role="dialog" aria-label="Choose date">
          <Calendar
            selected={fromISO(value)}
            onSelect={(d) => { onChange(toISO(d)); close(); }}
            min={minDate}
            max={maxDate}
            weekStartsOn={weekStartsOn}
            locale={locale}
            autoFocus
          />
        </div>
      )}
    </div>
  );
}
```

src/App.jsx

```
import { useState } from 'react';
import DatePicker from './components/DatePicker';
import { addMonths, startOfToday, toISO } from './utils/date';

const today = startOfToday();
const MIN = toISO(today); // no bookings in the past
const MAX = toISO(addMonths(today, 12));

export default function App() {
  const [date, setDate] = useState(null);

  return (
    <main className="app">
      <h1>Date picker</h1>
      <DatePicker label="Travel date" value={date} onChange={setDate} min={MIN} max={MAX}
weekStartsOn={1} />
      <p>Selected: <output>{date ?? 'none'}</output></p>
```

```
    </main>
  );
}
```

src/main.jsx

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import App from './App';
import './styles.css';

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
);
```

src/styles.css

```
* { box-sizing: border-box; }
body { margin: 0; font-family: system-ui, sans-serif; background: #f6f7fb; color: #1f2430; }
.app { max-width: 420px; margin: 48px auto; padding: 0 16px; }

.datepicker { position: relative; display: inline-flex; flex-direction: column; gap: 6px; }
.datepicker__field { display: flex; gap: 4px; }
.datepicker__field input { padding: 8px 10px; border: 1px solid #c5c9d6; border-radius: 6px; font:
inherit; width: 160px; }
.datepicker__field input[aria-invalid='true'] { border-color: #d33; }
.datepicker__field button { border: 1px solid #c5c9d6; background: #fff; border-radius: 6px; cursor:
pointer; padding: 0 10px; }
.datepicker__error { color: #d33; margin: 0; font-size: 13px; }
.datepicker__popover {
  position: absolute; top: calc(100% + 4px); left: 0; z-index: 10;
  background: #fff; border: 1px solid #dde0ea; border-radius: 10px; padding: 12px;
  box-shadow: 0 8px 24px rgb(0 0 0 / 0.12);
}

.calendar__header { display: flex; align-items: center; justify-content: space-between; margin-bottom:
8px; }
.calendar__header h2 { font-size: 15px; margin: 0; }
.calendar__header button { border: 0; background: none; font-size: 20px; width: 32px; height: 32px;
border-radius: 6px; cursor: pointer; }
.calendar__header button:disabled { opacity: 0.3; cursor: default; }

.calendar table { border-collapse: collapse; }
.calendar th { font-size: 12px; font-weight: 500; color: #6b7080; padding-bottom: 4px; }
```

```
.calendar td { padding: 1px; }

.day { width: 36px; height: 36px; border: 0; border-radius: 50%; background: none; font: inherit;
cursor: pointer; }
.day:hover:not(:disabled) { background: #eef0f7; }
.day:focus-visible { outline: 2px solid #3b5bdb; outline-offset: 1px; }
.day--outside { color: #a6aaba; }
.day--today { font-weight: 700; box-shadow: inset 0 0 0 1px #3b5bdb; }
.day--selected, .day--selected:hover:not(:disabled) { background: #3b5bdb; color: #fff; }
.day:disabled { color: #d0d3dd; cursor: not-allowed; text-decoration: line-through; }
```

src/utils/date.test.js

```
// Run with: node src/utils/date.test.js
import assert from 'node:assert/strict';
import { toISO, fromISO, addDays, addMonths, getMonthGrid, startOfWeek, clamp, isOutOfRange } from
'./date.js';

const d = (iso) => fromISO(iso);

// parsing
assert.equal(toISO(d('2026-11-02')), '2026-11-02');
assert.equal(fromISO('2026-02-31'), null);
assert.equal(fromISO('2026-13-01'), null);
assert.equal(fromISO('02/11/2026'), null);
assert.equal(toISO(d('2024-02-29')), '2024-02-29'); // leap year

// arithmetic across month / year boundaries
assert.equal(toISO(addDays(d('2026-12-31'), 1)), '2027-01-01');
assert.equal(toISO(addDays(d('2026-03-01'), -1)), '2026-02-28');
assert.equal(toISO(addMonths(d('2024-01-31'), 1)), '2024-02-29');
assert.equal(toISO(addMonths(d('2026-01-31'), 1)), '2026-02-28');
assert.equal(toISO(addMonths(d('2026-01-15'), -1)), '2025-12-15');
assert.equal(toISO(addMonths(d('2026-03-31'), 12)), '2027-03-31');

// grid: Nov 2026 starts on a Sunday
const nov = getMonthGrid(2026, 10, 0);
assert.equal(nov.length, 42);
assert.equal(toISO(nov[0]), '2026-11-01');
assert.equal(toISO(getMonthGrid(2026, 10, 1)[0]), '2026-10-26'); // Monday start
assert.equal(toISO(getMonthGrid(2026, 1, 0)[0]), '2026-02-01'); // Feb 2026 starts Sunday
assert.equal(toISO(startOfWeek(d('2026-11-04'), 1)), '2026-11-02');

// range
const min = d('2026-11-01'), max = d('2026-11-30');
assert.equal(isOutOfRange(d('2026-10-31'), min, max), true);
```

```
assert.equal(isOutOfRange(d('2026-11-30'), min, max), false);
assert.equal(isOutOfRange(d('2026-10-31'), null, null), false);
assert.equal(toISO(clamp(d('2026-12-05'), min, max)), '2026-11-30');

console.log('date utils: all checks passed');
```

---

12. Wrap-up

Edge cases handled

- Timezone off-by-one (string value, local parsing), month overflow, leap years, DST, year boundaries.
- Impossible dates rejected; typed input validated against format and range.
- `min` / `max` enforced for clicks, keyboard moves and month buttons.
- Focus never lost: roving tabindex, focus moves after render, returns to the trigger on close; Escape works even when focus fell to `<body>`.

Trade-offs to mention

- **No date library:** shows the math, ~50 lines. In production use `date-fns` (tree-shakable) for formatting, parsing in many formats and locale week starts.
- **Strict `YYYY-MM-DD` typing:** simple and unambiguous. Real users type `2/11/26`; parsing locale formats is a library job.
- **Not a full modal:** no focus trap; Tab-out closes the popover instead. Correct for a non-modal popover; a mobile full-screen sheet would trap focus.
- **Absolute positioning:** the popover can be cut off at the viewport edge or inside `overflow: hidden`. Fix with a portal + Floating UI.
- **Native `<input type="date">`** would give accessibility and mobile UI for free; custom UI is chosen for consistent design, ranges and custom disabled days.

What to add with more time

- **Range picker:** `{ start, end }` value, hover preview of the range, two months side by side.
- `isDateDisabled(date)` prop for holidays / sold-out days from an API (fetched per visible month, cached).
- Month / year dropdowns for far dates (date of birth).
- React Testing Library tests: keyboard paths, focus return, min / max clamping.
- Portal + collision-aware positioning; mobile bottom sheet.
- Localised input format and RTL support (ArrowLeft / Right swap in RTL).

---

13. Counter questions the interviewer may ask

Requirements

Q1. Why not just use `<input type="date">`?

It is accessible and gives a native mobile picker for free, and for a simple form I would use it. But its popup can't be styled, it looks different in each browser, and it can't show prices per day, ranges or custom disabled days. Booking sites need those, so they build a custom one.

Q2. Why store the value as a string instead of a `Date`?

A travel date is a calendar day, not an instant in time. A `Date` carries a timezone, and `new Date('2026-11-02')` is UTC midnight, which is Nov 1 in the US. A `YYYY-MM-DD` string is timezone-free, compares with `<`, serialises to JSON unchanged and matches what APIs expect.

Q3. What if the product needs date and time?

Then the value becomes an instant: store an ISO timestamp with offset (`2026-11-02T14:30:00+05:30`) or epoch ms, add a time selector, and decide whose timezone the time is in (user's or the hotel's). The calendar part stays the same.

State design

Q4. Why is there no `viewMonth` state?

It is derived from `focused`. With two states, paging to March while focus stays on a February day breaks keyboard navigation. One state keeps the visible month and the focused cell always in sync. ‹ › simply move `focused` by a month.

Q5. Why does `DatePicker` keep a separate `text` state?

A half-typed `2026-1` is not a valid value, so it can't live in the parent's `value`. `text` is a draft that is committed on Enter / blur and reset when `value` changes. I reset it during render (`if (value !== prevValue)`) rather than in an effect to avoid an extra render with stale text.

Q6. Why is the component controlled?

Like a native input, the parent decides the value: it can reset it, validate it or sync it to the URL. An uncontrolled version can be added with a `defaultValue` prop later; the reverse is harder.

Hard part 1: month grid

Q7. Why always 42 cells?

A month spans 4 to 6 weeks. A fixed 6-row grid keeps the popover height stable so the buttons don't jump under the mouse while paging. It also makes the grid shape a constant for tests.

Q8. How does `weekStartsOn` change the grid?

Only the offset: `(firstDay.getDay() - weekStartsOn + 7) % 7`. Weekday labels rotate the same way. Home / End use `startOfWeek(d, weekStartsOn)`, so keyboard navigation follows too.

Hard part 2: date math

Q9. What happens on Jan 31 when the user presses PageDown?

`addMonths` clamps the day to the target month's last day, so it goes to Feb 28 (Feb 29 in leap years) instead of overflowing to Mar 3. Pressing PageUp then returns to Jan 28, not Jan 31: the same trade-off every calendar makes.

Q10. Why not add `86400000` ms to move a day?

On a DST change a day is 23 or 25 hours, so adding 24 hours can land on the same date or skip one. `new Date(y, m, d + 1)` lets the engine handle it in local calendar terms.

Hard part 3: keyboard and accessibility

Q11. What is a roving tabindex and why use it?

Only one cell (the focused day) has `tabIndex=0`; the rest have `-1`. Tab enters the grid on one day and leaves in one step instead of 42; arrows move inside. It is the ARIA grid pattern used by WAI-ARIA's date picker example.

Q12. Why move focus in an effect and not in the key handler?

When the move crosses a month, the target button only exists after React re-renders the new grid. The effect runs after commit, when it is in the DOM. The ref flag makes sure only keyboard moves steal focus, not clicks on ‹ ›.

Q13. How does a screen reader user know what day they are on?

Each button has `aria-label` with the full date from `Intl` ("Monday, 2 November 2026"), the selected cell has `aria-selected`, today has `aria-current="date"`, the title has `aria-live="polite"` so month changes are announced, and the trigger has `aria-haspopup` / `aria-expanded`.

Hard part 4: popover

Q14. Why `pointerdown` on `document` instead of a `blur` handler?

Clicking a non-focusable area inside the popover (the title, padding) also blurs the input, so a blur-only close would close it wrongly. `pointerdown` + `ref.contains(target)` checks where the click actually happened, and covers mouse, touch and pen. A focus-out check with `relatedTarget` handles Tab-out separately.

Q15. Why is the Escape listener on `document`, not on the popover?

On the earliest allowed month, clicking ‹ disables that button; a disabled button can't keep focus, so focus drops to `<body>` and the popover's own `onKeyDown` never fires. A document listener active only while open always catches it.

Implementation steps and performance

Q16. Do you need `React.memo` on the cells?

No. 42 buttons re-rendering on a key press is trivial. I memoise the grid array per month and avoid creating `Intl.DateTimeFormat` per cell (one formatter per render). Memo would add props comparison cost for no measurable gain.

Q17. What if each day must show a price from an API (MakeMyTrip fare calendar), and the API is slow?

Fetch per visible month keyed by `YYYY-MM`, cache results in a `Map` (or React Query), prefetch the next month, and abort the old request with `AbortController` when the user pages quickly. Render the grid immediately with skeleton prices so navigation never waits on the network.

Q18. What if you need a range of 1 million disabled dates?

Don't pass an array to check with `includes` per cell. Pass a `Set` of ISO strings (O(1) lookup) or, better, rules / intervals (`isDateDisabled(date)` , sorted ranges + binary search). The calendar only ever asks about 42 dates at a time.

Trade-offs

Q19. How would you extend this to a range picker?

Value becomes `{ start, end }` . First click sets `start` , second sets `end` (swap if earlier), hover shows a preview range, cells get `in-range` / `range-start` / `range-end` classes. The utils, grid and keyboard code are reused unchanged; only selection logic in `DatePicker` changes. Often two months are shown side by side.

Q20. The popover is cut off at the bottom of the screen. How do you fix it?

Render it in a portal to `document.body` and position it with Floating UI (`flip` , `shift`), which also handles scroll containers and `overflow: hidden` parents. On mobile, switch to a bottom sheet.

Q21. How would you test this?

Unit tests for `utils/date.js` (leap years, overflow, grid start, parsing). React Testing Library + `user-event` for: open → focus on today, arrows across months, PageDown on Jan 31, min / max clamping, Escape returns focus, typed invalid text shows the error. Freeze "today" with fake timers so tests don't depend on the run date.

Q22. What changes for RTL languages?

The grid is mirrored by CSS (`dir="rtl"`), so ArrowLeft must mean "next day". Swap the ArrowLeft / ArrowRight handlers when `dir` is `rtl` , and flip the ‹ › icons.